Course Project Part 1

Name: Faith Gibbs

ASUID: 1225851382
Course: CSE 464

GitHub Repository Link:
https://github.com/speedylight61-del/CSE464-2026-frgibbs.git

Project Overview

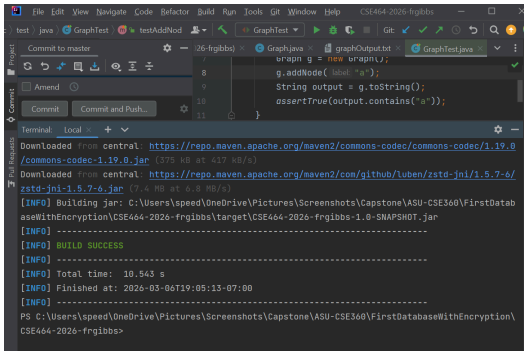
In this project, I created a Java program that reads a graph written in DOT format and builds a graph structure. The program reads nodes and edges from an input file, allows additional nodes and edges to be added, and then outputs the graph to a new file.

I developed this project using Java, Maven, and IntelliJ. The program demonstrates graph parsing, node insertion, edge creation, and exporting the graph in DOT format.

How to Compile and Run the Program

This project uses Maven.

- Step 1: Clone the repository
    - git clone [PASTE YOUR GITHUB REPO LINK]
- Step 2: Navigate into the project folder
    - cd CSE464-2026-frgibbs
- Step 3: Compile and build the project
    - mvn package



- Step 4: Run the program

- Run the Main class inside IntelliJ or run the compiled program.

Example Input

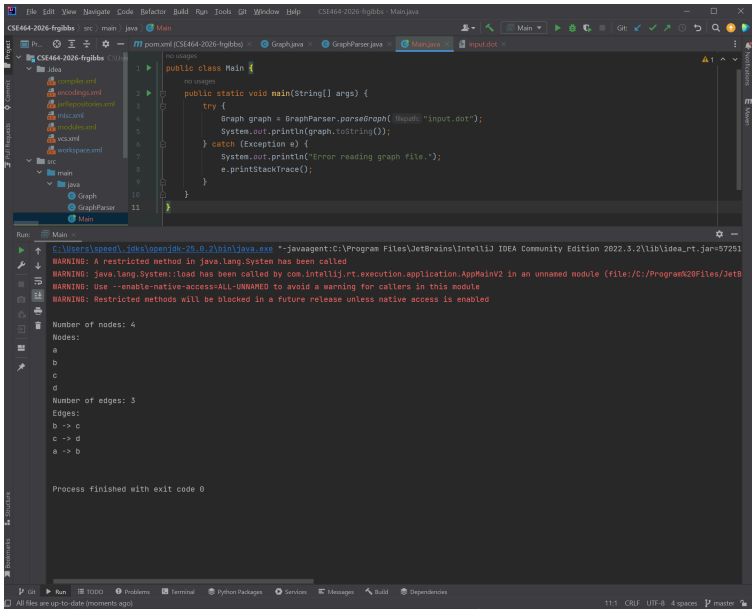
- The program reads a DOT file called input.dot.
- Example input file:
  - digraph {
    a -> b;
    b -> c;
    c -> d;
    }
- This input describes a directed graph with three edges.

Feature 1 : Parse Graph

First, I implemented the ability for the program to read the DOT file and extract the nodes and edges. The graph information is stored in sets for nodes and edges. After parsing the file, the program prints the nodes and edges to the console.

- Example Output:
  - Number of nodes: 4
    Nodes:
    a
    b
    c
    D
    Number of edges: 3
    Edges:
    a -> b
    b -> c
    c -> d

Screenshot of Feature 1 Output:



GitHub Commit Link:
https://github.com/speedylight61-del/CSE464-2026-frgibbs/commit/315b46b2a8ffc00a184465870beeeb2547a995b7

Feature 2 : Add Nodes

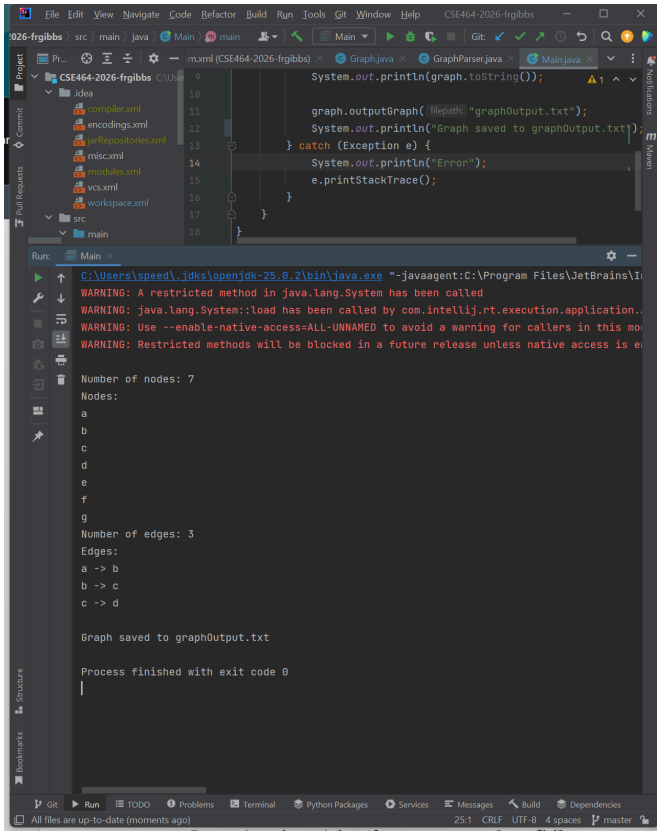
Next, I implemented functionality to add nodes to the graph.

- The graph supports two methods:
  - addNode() – adds a single node
  - addNodes() – adds multiple nodes at once
- Example code:
  - graph.addNode("e");
  - graph.addNodes(new String[]{"f","g"});

After adding nodes, the graph contains additional nodes.

- Example Output:
  - Number of nodes: 7

Screenshot of Feature 2 Output:



GitHub Commit Link:
https://github.com/speedylight61-del/CSE464-2026-frgibbs/commit/dab0799926cb21652037c2ac08f821600c5b1998

Feature 3 : Add Edges

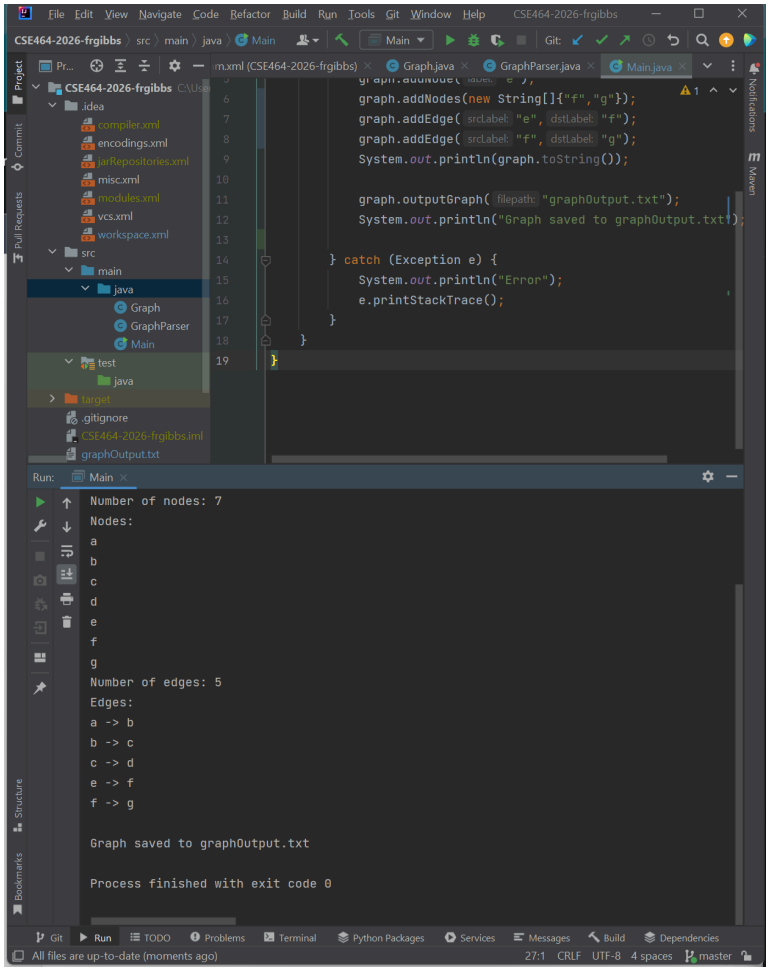
After that, I added functionality to create new edges between nodes.

- Example code:
  - graph.addEdge("e","f");
  - graph.addEdge("f","g");

This creates two new edges connecting the new nodes.

- Example Output:
  - Number of edges: 5

Screenshot of Feature 3 Output:



GitHub Commit Link:
https://github.com/speedylight61-del/CSE464-2026-frgibbs/commit/092fc9146bb80e276b097a09639415fc7edd3365

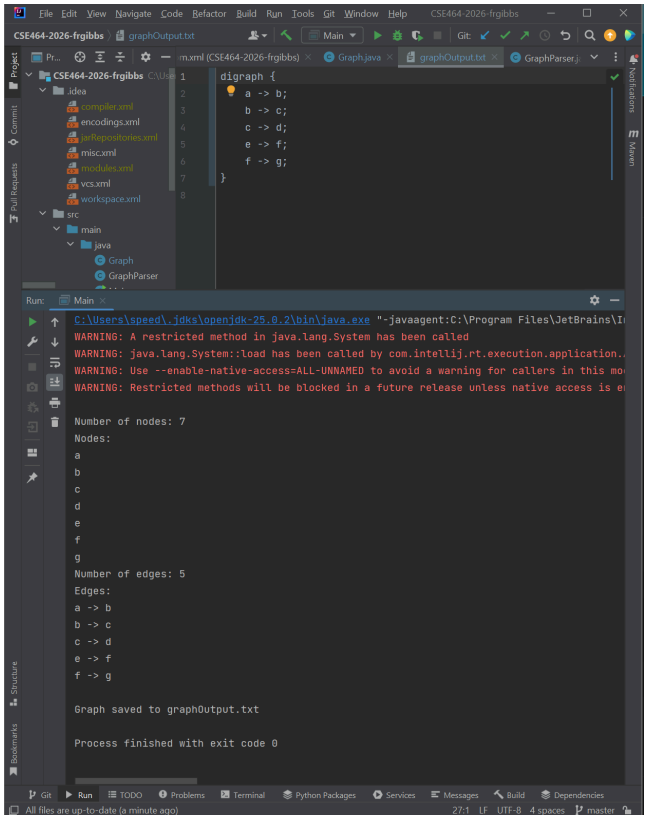
Feature 4 : Output Graph

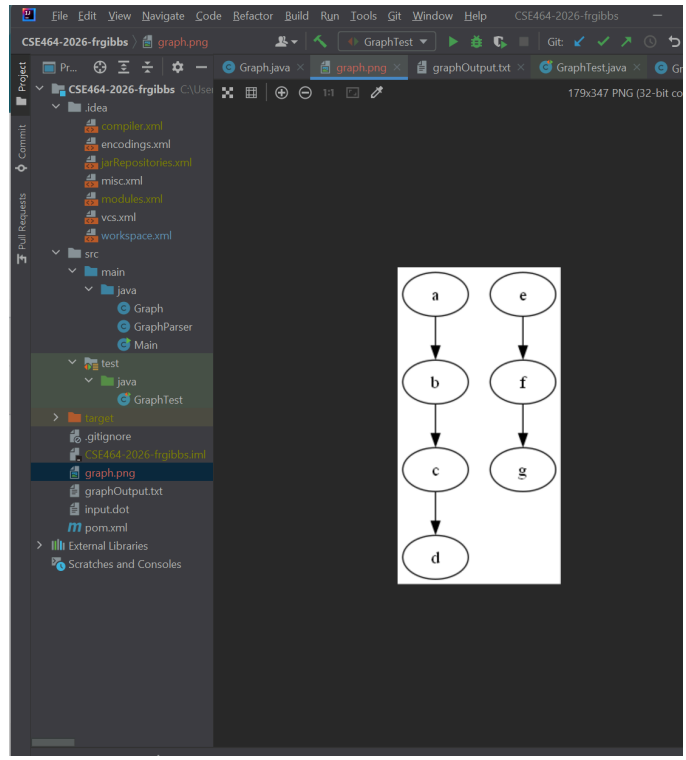
I also implemented functionality to export the graph to a file in DOT format.

- Example code:
  - graph.outputGraph("graphOutput.txt");
- Example output file:
  - digraph {
    a -> b;
    b -> c;
    c -> d;
    e -> f;

```
        f -> g;
    }
```

Screenshot of Output File:





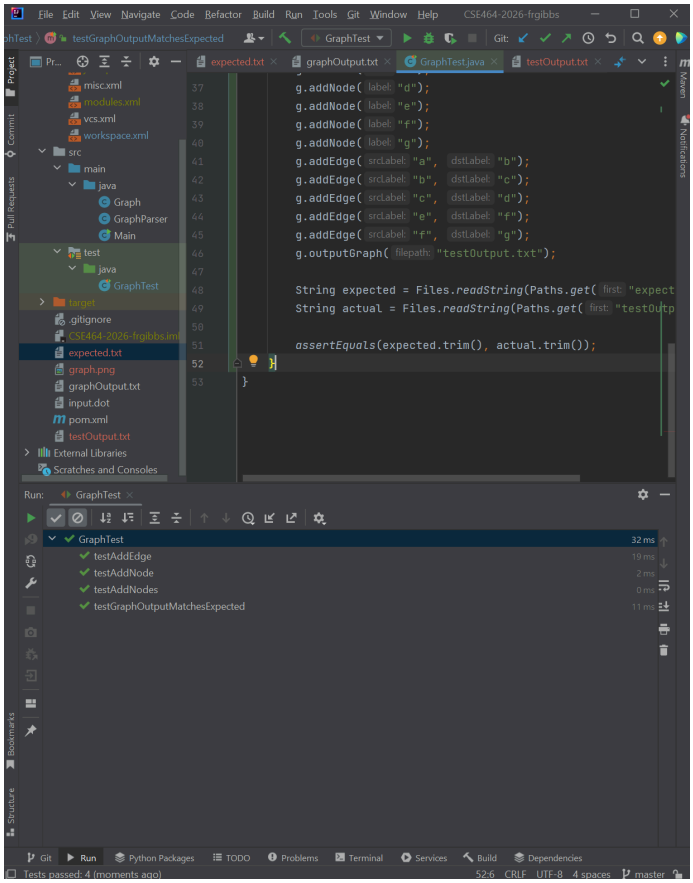
GitHub Commit Link:
https://github.com/speedylight61-del/CSE464-2026-frgibbs/commit/89d08b9a979a31eb6621e5827189b0c3dde9e4ad

Unit Tests

I created JUnit tests to verify that nodes and edges are added correctly. These tests confirm that the graph contains the expected nodes and edges after operations are performed.

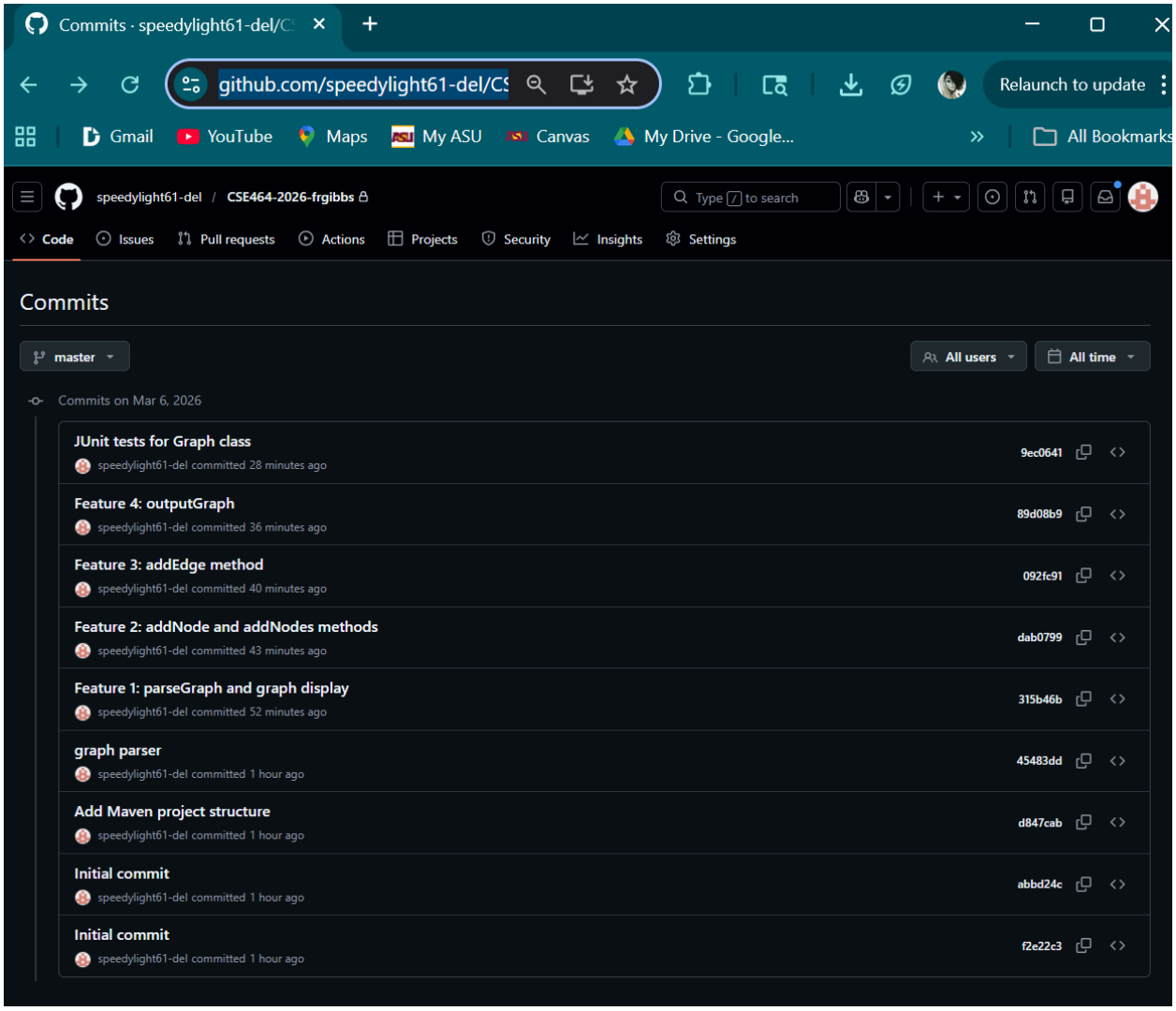
- Example tests include:
    - testAddNode()
    - testAddNodes()
    - testAddEdge()

Screenshot of Passing Tests:



GitHub Commits

I committed each feature separately to GitHub so the development process can be clearly tracked.

- The commits include:
  - Feature 1 – Parse Graph
  - Feature 2 – Add Nodes
  - Feature 3 – Add Edges
  - Feature 4 – Output Graph
  - Unit Tests

Screenshot of GitHub Commit History: